Day 3 — Mapping High-Level System Components

> **Activity packet** for your triad. Today's job: draw a **C4-style component diagram** for your feature — Context (the world around the system) and Container (the deployable units inside it). The diagram replaces hand-waving with a shared visual vocabulary.

Where we are in the week

Day 1 produced the architecture stance + integration table. Day 2 produced the threat model + revised security NFRs. Today produces **TCD Section 2** (now upgraded with a real diagram) and supports Section 3 by clarifying the security boundaries.

By 16:00, every triad has two C4 diagrams ready to walk through with an architect.

Inputs

- TCD Sections 1–3
- The integration table (Day 1)
- The data-flow sketch (Day 2 STRIDE pass)
- The PRD's Section 5 solution sketch and Section 10 dependencies

---

C4 — the four levels (and why we use only two)

The C4 model is a layered diagramming approach by Simon Brown:

Level	What it shows	Audience	We draw it?
Context	The system + its users + neighbors	Everyone, including stakeholders	✅ Today
Container	Deployable units inside the system	TPMs, architects, devs	✅ Today
Component	Code-level building blocks within a container	Engineers	❌ Engineering job
Code	Class-level detail	Engineers	❌ Engineering job

Today we draw **Context + Container**. That's TPM scope.

The C4 model deliberately avoids cluttering with too many notations. We use simple boxes, labeled arrows, and a legend.

The Context diagram (level 1)

The Context diagram answers: *who interacts with this system, and what other systems does it talk to?*

A FieldPulse reconcile-flow Context diagram includes:

- **People** (boxes with stick figures or labels): Dispatcher, Field Tech, Operations VP, FieldPulse Admin
- **The system in scope**: "FieldPulse Reconcile Flow" (one big box; we explode it next)
- **External systems**: SSO Identity Provider, Audit Event Store, Tickets API, Payments / Timecard System, Notification Service

Arrows show who calls whom, with brief labels: "submits reconcile" / "issues SSO token" / "writes audit events".

What's deliberately NOT in the Context diagram:

- Database choices
- Internal services
- Protocols (REST vs gRPC) — labels, not the focus
- Hostnames, environments, deployment units

Keep it under 12 boxes. If it has 20, you're sneaking Container concerns in.

The Container diagram (level 2)

The Container diagram answers: *inside the system, what are the deployable units, and how do they talk to each other?*

A "container" in C4 is **anything you would update independently** — a web app, a mobile app, a service, a database, a queue, a static-site CDN, an event bus.

For FieldPulse reconcile inside a modular monolith:

- **Mobile app** (containers: native Android, native iOS — both deployables)
- **Backend monolith** (one Spring Boot service, with reconcile module inside)
- **Database** (Postgres primary + read replica)
- **Audit event bus** (Kafka topic shared with platform)
- **Object store** (S3 for the rare reconcile-attachment case)

Arrows show the protocol and direction, with brief intent labels.

The container diagram is where the architecture stance from Day 1 becomes visible. A modular monolith looks like one big container with internal modules listed; a microservice split looks like multiple containers.

The legend (do not skip)

Every C4 diagram needs a legend so non-architects can read it:

```
Legend
- □ Box with no fill = system / module owned by us
- ■ Box with fill = external system (out of our control)
- 👤 Stick figure / labeled box = person (role)
- → Solid arrow = synchronous call
- ⇢ Dashed arrow = async / event
- (label on arrow) = intent in plain English
```

A diagram without a legend is a Rorschach test. Force the legend.

Activity 1 — Context Diagram

Format: Triad • **35 min** • Block 1

Purpose

Draw the Context diagram for your PRD feature. It should be readable in 60 seconds by a stakeholder who has never seen the feature.

Setup

Each triad needs whiteboard or large paper, markers, and the integration table from Day 1. AI optional; log provenance if used.

Triad protocol

1. **List people** (5 min). All roles that interact with the feature. Include indirect roles (e.g., the Operations VP who reviews reconciles).
2. **List external systems** (5 min). Pull from the integration table. If you find new ones today, add them to the table now.
3. **Draw it on a whiteboard or paper** (15 min). One central box ("the system"); people on the left; external systems on the right or below. Label each arrow with the intent.
4. **Add the legend** (5 min). Don't skip it.
5. **The 60-second test** (5 min). One member walks an "outsider" (another triad) through the diagram in 60 seconds.

What "good" looks like

- Under 12 boxes total
- Every arrow has a verb-based label ("submits reconcile", not just "POST")
- Indirect roles are present (the people who don't *use* the feature but care about it)
- A stakeholder who has never seen the feature can ask sensible questions after 60 seconds

Deliverable

A Context (C4 Level 1) diagram for the feature with people, external systems, labeled arrows, and a legend — passing the 60-second outsider test.

Activity 2 — Container Diagram

Format: Triad • **40 min** • Block 2

Purpose

Draw the Container diagram. Every container is something you'd deploy, update, or scale independently.

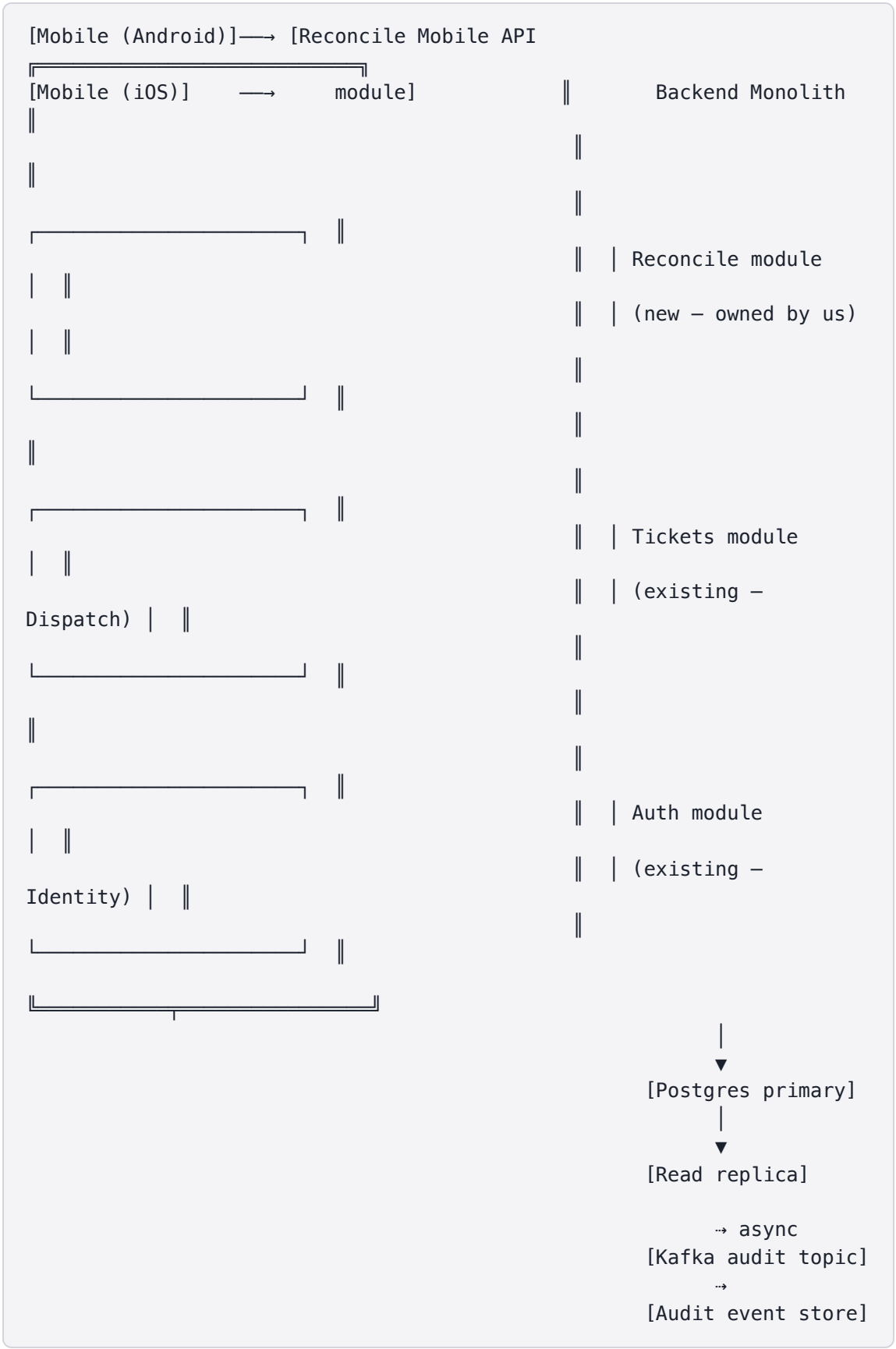
Setup

Each triad needs the Context diagram from Activity 1, the architecture stance from Day 1, and large paper or whiteboard space for containers + modules.

Triad protocol

1. **List containers** (10 min). For each, name what it is, what tech stack (rough — "Spring Boot", "iOS native", "Postgres", not framework-version detail), and who owns it.
2. **Draw the diagram** (15 min). Containers as boxes; arrows with protocol labels.
3. **Annotate the modular split** (10 min). If the architecture stance from Day 1 was "modular monolith", draw the modules **inside** the monolith container as labeled sub-boxes. Don't promote them to standalone containers.
4. **The 90-second test** (5 min). One member walks another triad through the diagram, including: which containers we own, which we depend on, which arrows are sync vs async.

Worked example — modular monolith Container diagram



Output

A Container diagram that an architect can react to. Even (especially) a hand-drawn paper version works — clarity beats prettiness.

Activity 3 — Stress-Test the Diagram

Format: Triad-pair • **40 min** • Block 3

Purpose

Pair triads. Each triad's diagram is read by another triad. Find what's missing, what's wrong, what's unclear.

Setup

Instructor pairs triads. Each pair needs both triads' Context and Container diagrams plus the three-lens prompt card.

The three lenses

The reviewer triad reads the diagram with these three lenses:

Lens	Question
Failure	Trace a failure: "What happens when the audit topic is full?" Walk the diagram.
Trust boundary	Where does data cross from our control to someone else's? Mark with a thick line.
Evolvability	If the architecture stance changes (modular monolith → microservice), which arrows would have to change?

Triad-pair protocol

1. **Swap diagrams** (5 min). Spend 5 min reading the other triad's diagrams.
2. **Failure trace** (10 min). Pick one failure scenario; walk the diagram aloud. The author triad listens.
3. **Trust boundary mark-up** (10 min). Reviewer marks where data crosses out of the author's control.
4. **Evolvability question** (10 min). Reviewer asks "if you were to split X into a new service, which arrows would change?"
5. **Author updates** (5 min). Authors annotate their diagram with the findings.

What "good" looks like

- Trust boundaries are explicit — diagrams without them hide compliance and security risks.
- The failure trace exposes a missing arrow or a missing failure-handling stance.
- The evolvability conversation produces 1–2 specific arrows to watch as the architecture matures.

Deliverable

Annotated diagrams with trust boundaries marked, a failure trace recorded, and 1–2 evolvability arrows noted.

Activity 4 — AI-Assisted Diagram Critique + Final Polish

Format: Triad • **45 min** + Wrap • Block 4

Purpose

Use AI as a critic to surface what a senior architect would push back on. Update the diagram and TCD Section 2.

Setup

Each triad needs both diagrams in text form (for prompt input), the TCD Section 1 stance, and the two prompts. AI required; log provenance.

The two prompts

A. "What's wrong with this diagram?"

```
Role: Principal architect reviewing a feature's C4 Container diagram.
Context: <description of the diagram in plain text:
  containers + their tech + arrows + protocols + who owns each>
Stance: <copy from TCD Section 1>
Task: Identify the top 3 issues in the diagram or the architectural
stance
      it implies. For each, name the specific scenario where the issue
would matter.
Constraints:
  - Treat ownership and tech choices as given, not as targets
  - Flag any unstated assumption
  - Do not suggest generic 'modernization'
Format: Numbered list, each with: Issue / Scenario / Suggested
clarification.
```

B. "What would a stakeholder ask?"

```
Role: Operations VP at a B2B SaaS company.
Context: <same diagram description>
Task: List 5 questions you would ask after seeing this diagram in a
review meeting.
Constraints:
  - Questions should be the kind a non-engineer would actually ask
  - Avoid framework-level detail
Format: 5 numbered questions, each with the concern behind it.
```

Triad protocol

1. **Run Prompt A** (10 min). Capture top 3 issues.
2. **Decide which to address in the diagram** (10 min). Adopt / defer / reject — same Week 3 discipline.
3. **Run Prompt B** (10 min). Capture stakeholder questions.
4. **Update the diagrams** (10 min). Final polish, including legend, trust boundaries, and key annotations.
5. **Provenance note** (5 min). What prompts, what was adopted, what was rejected.

Deliverable

Polished Context + Container diagrams with legend, trust boundaries, stakeholder-question list, and AI-prompt provenance note appended to TCD Section 2.

Wrap (last 15 min)

Each triad shares:

- One thing the diagram **made explicit** that the integration table didn't
- One thing the diagram **revealed was wrong** about their original assumptions
- One question the diagram couldn't answer (likely needs the architect)

End-of-day checkpoint

Each triad ends Day 3 with:

- [x] **Context diagram** with people, the system, external systems, and a legend
- [x] **Container diagram** showing deployables, protocols, and modular structure
- [x] **Trust boundary** marked
- [x] **Failure-trace finding** integrated
- [x] AI-prompt provenance note for today

- [x] TCD Section 2 updated to reference the diagrams